

TrainTimetable - opis projektu baza i aplikacja

Tematyka

Projekt dotyczy systemu informatycznego dla rozkładu jazdy pociągów pasażerskich. Zakres obejmuje bazy danych i aplikacje webowa do obsługi wyszukiwania połączeń. Model danych odwzorowuje rzeczywistą strukturę przewozów: stacje, postoje, trasy i składy. Dodatkowo uwzględniono kontekst administracyjny i geograficzny stacji. W projekcie nacisk położono na spójność danych i praktyczne scenariusze użytkownika.

Cele

Celem głównym było zaprojektowanie relacyjnej bazy co najmniej w 3 postaci normalnej. Drugi cel to zapewnienie integralności przez klucze, ograniczenia i walidacje biznesowe. Trzeci cel obejmował udostępnienie aplikacji CRUD zgodnej z tematyką kolejową. System miał obsługiwać zarówno kursy cykliczne, jak i przejazdy dla konkretnych dat. Wymaganiem praktycznym było także wyszukiwanie połączeń z przesiadkami. Istotny był też aspekt taboru: skład pociągu, typy wagonów i miejsca pasażerskie.

Czytelny opis schematu bazy

Schemat składa się z 17 tabel. Częściowo dzieli się na obszar geograficzny i operacyjny. Obszar geograficzny tworzą tabele WOJEWODZTWA, POWIATY, GMINY, STACJE. Relacja hierarchiczna idzie od województwa do stacji przez powiat i gminę. Tabela INFRASTRUKTURA_STACJI przechowuje perony i tory dla konkretnej stacji. UNIQUE dla id_stacji, numer_peronu, numer_toru zapobiega dublowaniu infrastruktury. Obszar taborowy obejmuje TYPY_WAGONOW, ELEMENTY_STALE, MIEJSCA, WAGONY. TYPY_WAGONOW definiują parametry geometrii i charakter wagonu. ELEMENTY_STALE i MIEJSCA opisują układ wewnętrzny każdego typu wagonu. WAGONY to instancje fizyczne, które później są przypinane do pociągów i tras. Obszar ruchu pociągów tworzą POCIAGI, SKLADY, TRASY, PRZEJAZDY, TRASY_CYKLICZNE, POSTOJE. SKLADY łączy pociąg i wagon oraz pilnuje kolejności wagonów w składzie. TRASY przechowują opis kierunku i domyślnie przypisany pociąg. PRZEJAZDY i TRASY_CYKLICZNE rozdzielają harmonogram datowy i tygodniowy. POSTOJE przechowują kolejność punktów trasy oraz czasy przyjazdu i odjazdu. Dzień offset w POSTOJE obsługuje przejście przez północ i kursy wielodniowe. Rozszerzenie operacyjne zapewnia SKLADY_SEGMENTY i ZMIANY_SKLADU. Dzięki temu model obsługuje odpinanie i dopinanie wagonów na wybranych stacjach. W bazie zastosowano klucze obce, CHECK, UNIQUE oraz indeksy wydajnościowe. Indeksy obejmują między innymi POSTOJE, INFRASTRUKTURA_STACJI i PRZEJAZDY. Takie podejście poprawia czas wyszukiwania tras i budowania harmonogramów.

Napotkane problemy i rozwiązania

Pierwszym problemem była walidacja godzin na pierwszym i ostatnim postoju trasy.

Rozwiązaniem było rozdzielenie logiki przyjazdu i odjazdu oraz wymuszenie pustych pól.

Drugim problemem były kursy przekraczające północ i zaburzenia porównywania czasu.

Rozwiązano to przez przechowywanie offsetu dnia dla przyjazdu i odjazdu.

Trzecim problemem było mieszanie harmonogramu cyklicznego z jednorazowym.

W modelu i walidacji wymuszono rozłączne podejście do tych trybów kursowania.

Kolejny problem dotyczył integralności składu przy przepinaniu wagonów między trasami.

Wprowadzono tabele segmentów składu i historii zmian oraz logikę przywracania.

Wyzwanie stanowiła też czytelność błędów SQL dla administratora.

W aplikacji dodano mapowanie komunikatów bazowych na bardziej zrozumiałe opisy.

Informacje o aplikacji

Aplikacja została wykonana w Pythonie z wykorzystaniem Flask i Flask SQLAlchemy.

Punkt startowy to `app.py`, konfiguracja łącza z bazą znajduje się w `config.py`.

Warstwa modeli w `models.py` odwzorowuje strukturę tabel i zależności relacyjne.

Główna logika biznesowa i endpointy HTTP znajdują się w `routes.py`.

Interfejs użytkownika i panel administratora są zrealizowane jako szablony HTML.

Aplikacja pracuje lokalnie i korzysta z bazy PostgreSQL `kolei_db`.

Funkcjonalności

Moduł użytkownika pozwala wyszukać połączenia między stacjami dla daty i godziny.

Algorytm wyznacza trasy bezpośrednie oraz warianty z jedną lub dwiema przesiadkami.

Wyniki pokazują czasy, stacje przesiadkowe, kategorie i nazwy pociągów.

Dostępny jest podgląd szczegółów przejazdu wraz z harmonogramem postojów.

Pokazywana jest też struktura wagonów i miejsca specjalne w zadanym odcinku trasy.

Panel admina obsługuje dodawanie, edycje i usuwanie tras oraz harmonogramów.

Administrator zarządza wagonami i typami składu dla pociągów.

Dostępny jest proces przepinania wagonów między trasami na wspólnej stacji.

Interfejs API udostępnia pomocnicze dane o infrastrukturze i możliwych trasach.

Instalacja i użytkowanie

1. Utworzyć użytkownika i baze przez skrypt `baza_danych_login.sql`.

2. Utworzyć strukturę i dane uruchamiając `baza_danych_create.sql`.

3. Zainstalować zależności poleceniem `pip install -r requirements.txt`.

4. Uruchomić aplikację poleceniem `python app.py`.

5. Otworzyć przeglądarkę i wejść na stronę główną pod adresem `localhost:5000`.

Tryb użytkownika służy do wyszukiwania połączeń i analizy szczegółów podróży.

Tryb admin służy do utrzymania danych operacyjnych i konfiguracji tras.

Do resetu struktury można użyć skryptów `drop.sql` oraz ponownie `create.sql`.

Podsumowanie

Projekt spełnia wymagania etapu z bazą i aplikacją, zachowuje spójny model danych, zapewnia praktyczne funkcje dla użytkownika i administratora oraz możliwość rozwoju.